Compile Yourself

> *In the basement of an old research lab, behind a stack of dead terminals, you find a computer that someone left running. Its screen shows nothing, and its memory is full of zeros. It has a keyboard, a printer, and exactly one instruction: subtract, and jump if the answer is negative.*
>
> *Taped to the case is a note from the last engineer who used it. "It can't add. It can't loop. It can't call a function. I taught it all of that, then I taught it to compile a language, and then I taught it to compile itself."*
>
> *The manual is gone, and so is the engineer. You have five days. How far can you recover the engineers work?*

The real history is not far from this story. The first C compiler was written in assembly, and once it worked, C was rewritten in C. You will retrace that climb on a machine with a single instruction.

There are three tiers. **Tier 1** is to build the toolkit the machine is missing. **Tier 2** is to write a compiler for a small language called v1. **Tier 3** is to write a v1 compiler in v1 itself.

The machine

- **Memory:** 1,048,576 words (call this N), at addresses 0 to N-1. Each word is a signed 32-bit integer, and arithmetic wraps around on overflow.
- Your program is loaded at address 0. Every other word starts at 0.
- **One instruction,** three words long: `a b c`. Starting at address `pc`, one step does this:

```
r = mem[a] - mem[b]
mem[a] = r
if r < 0:  pc = c
otherwise: pc = pc + 3
```

- The machine starts with `pc = 0`. Each executed instruction counts as one step.
- **Halt:** the machine halts normally when `pc` is outside `0 ... N-3` after a step. Jumping to `-1` is the usual way to stop.
- **Input:** address N-2 (1,048,574) is `IN`. Reading it as `b` returns the next input byte (0 to 255), or `-1` once the input has ended. Each read consumes one byte.
- **Output:** address N-1 (1,048,575) is `OUT`. It always reads as 0. If `a = OUT`, the machine prints the low 8 bits of `r` (here `r = 0 - mem[b]`) and stores nothing. The branch still uses `r`.
- **Errors:** using `IN` as `a`, or any `a` or `b` outside `0 ... N-1`, stops the program with an error.
- **Code and data share memory.** Instructions are read from memory when they run, so a program may modify itself.
- A test passes only if the program halts normally within the step limit and its output matches the expected output byte for byte.

Provided tool

- `./vm program.txt < input.txt` runs a program. A program file is whitespace-separated decimal integers (negative numbers allowed), loaded from address 0. The machine's output goes to standard output.

There is no assembler. You write numbers, and how you get them is up to you.

Here is a complete program:

```
1048575 6 3  7 8 -1  -65 -1 0
```

It prints `A` and halts. Read it as three words per instruction, then data. The first instruction, `1048575 6 3`, has `a = OUT` and `b = 6`. Address 6 holds -65, so the machine prints `0 - (-65) = 65`, which is `A`. The second instruction, `7 8 -1`, subtracts address 8 (holding 0) from address 7 (holding -1) and gets -1, which is negative, so it jumps to -1 and halts. Addresses 6, 7, and 8 hold the data `-65 -1 0`.

Tier 1: Build the toolkit

The machine has no move, add, jump, pointer access, or call, and there is no assembler either. Build what you need, in any language. Then write these four programs as machine code. Only the final program files are tested.

- **ECHO:** copy the input to the output.
- **REVERSE:** the input is up to 1000 bytes (no NUL bytes). Output the same bytes in reverse order.
- **BIGMUL:** the input is two lines, each a non-negative decimal integer of up to 40 digits (no leading zeros, except the number 0 itself), each followed by a newline. Output their product in decimal, followed by a newline.
- **HANOI:** the input is `n` followed by a newline, with 1 <= n <= 8. Output the moves of the standard recursive Towers of Hanoi solution, one per line, written as the source peg followed by the destination peg (for example `AC`). The disks start on peg `A`, end on peg `C`, and peg `B` is the spare.

Each program is tested on visible and hidden inputs, with a limit of 100,000,000 (10^8) steps per run.

Tier 2: A compiler for v1

Write a compiler from the language **v1** to this machine. Your compiler can be written in any language, and how you build and run it is up to you. It must run as a single command that reads a v1 program on standard input and writes the machine program (whitespace-separated decimal integers, the format `vm` reads) to standard output.

The language v1

- **Structure.** A program is a list of top-level declarations: `var g;` (a global integer), `array a[100];` (a global array; the size is a positive decimal literal), and `func f(p, q) { ... }` (a function with 0 to 8 integer parameters).
- Execution starts at `main()`, which takes no parameters. The program halts normally when `main` returns.
- A function must be defined before it is used. A function may call itself.
- Comments start with `//` and run to the end of the line.
- Names are letters, digits, and underscores, and do not start with a digit. They are case-sensitive and at most 24 characters long. The keywords are `var array func if else while return`. `getc` and `putc` are built in.
- A source file is at most 60,000 bytes and has at most 200 functions.
- **Statements.** Function bodies are blocks in braces. Braces are always required.
- `var x;` declares a local variable (declare it before using it).
- `x = e;` and `a[e] = e;` are assignments.
- `f(e, ...);` is a call used as a statement.
- `if (e) { ... }`, with an optional `else { ... }`. There is no `else if`; write `else { if ... }` instead.
- `while (e) { ... }` and `return e;`. There is no `for`, no `break`, and no bare `return;`.
- **Expressions.** Decimal literals (up to 2147483647), variables, `a[e]`, calls `f(e, ...)`, unary `-e`, `*`, `+`, `-`, the comparisons `< <= > >= == !=`, and parentheses.
- From tightest to loosest: unary minus, then `*`, then `+` and `-`, then the comparisons. All binary operators are left-associative.

- A comparison yields 1 or 0. In `if` and `while`, any nonzero value counts as true.
- Operands and call arguments are evaluated from left to right.
- **Meaning.** All values are 32-bit signed integers and arithmetic wraps around. There is no division, no strings, and no `&&`, `||`, or `!`.
- Global variables and global arrays start at 0. Local variables have no defined starting value, so assign them before you read them. Parameters are local variables and may be assigned. A local or parameter hides a global of the same name.
- A function that ends without a `return` returns 0.
- `getc()` returns the next input byte (0 to 255), or -1 at the end of the input. `putc(e)` prints the low 8 bits of `e` and returns 0.
- Recursion goes at most 1,000 calls deep. Using an array index outside the array, or going deeper than that, is undefined.
- Comparisons < <= > >= compare the true signed values of their operands, even when the difference of the two would overflow 32 bits. For example, 2000000000 < -2000000000 is 0, and -2000000000 < 2000000000 is 1.

Two examples

```
func fact(n) {
    if (n < 2) { return 1; }
    return n * fact(n - 1);
}
func main() { putc(48 + fact(3)); putc(10); }   // prints "6"
```

```


var total;
array sq[5];

func square(x) { return x * x; }

func main() {
    var i;
    i = 0;
    while (i < 5) {
        sq[i] = square(i + 1);
        total = total + sq[i];
        i = i + 1;
    }
    if (total == 55) { putc(111); putc(107); }   // 1+4+9+16+25 = 55
    putc(10);
}   // prints "ok"
```

How Tier 2 is tested. Your compiler is run on hidden v1 programs, each at most 20,000 bytes. Each compiled program is run on vm with the test's input. Compiling must take under 60 seconds. Each run is limited to 100,000,000 (10^8) steps. A compile error on a valid program counts as a failed test.

Tier 3: Compile yourself

Write a compiler for v1 **in v1**. Your file `tier3.v1` is a v1 program that reads a v1 program from its input (until the end of input) and prints the compiled machine program to its output, in the same whitespace-separated format `vm` reads. It must follow every rule of v1, because your Tier 2 compiler will compile it. It must handle the whole language, and inputs of up to 60,000 bytes, which includes its own source.

It is checked in three steps:

- **Step 1.** We compile `tier3.v1` with your Tier 2 compiler. Call the result C1.
- **Step 2.** We run C1 on `vm` with hidden Tier 2 programs as input, then run each program it prints exactly as in Tier 2. Each one must give correct output.
- **Step 3 (the bootstrap).** We run C1 with `tier3.v1` as its input. Call its output C2. Then we run C2 with `tier3.v1` as its input, and call that output C3. C2 and C3 must be identical: the compiler, compiled by itself, reproduces itself.

Each run of your compiler on the machine is limited to 10,000,000,000 (10^10) steps.

Submission

Submit a repository with real commit history, containing:

- `tier1/` with the four machine programs (`echo.txt`, `reverse.txt`, `bigmul.txt`, `hanoi.txt`).
- `tier2/` with your compiler and a `BUILD.md` that says how to build and run it.
- `tier3/tier3.v1`.
- Any tools you built, and a one-page design log of what you tried and what failed.

Evaluation

Your submission is first run against the tests. After that, each team gives a **live demo** of its work and answers questions about it. Be ready to run your programs and your compiler for us, to explain how your pointer access and function calls work, and to adapt to a changed rule on the spot. Results are provisional until the demo is done. Work that a team cannot explain may be disregarded.